

Hands-on Lab: Building a LangChain AI Agent

Cohere + DuckDuckGo + Wikipedia + Custom Calculator

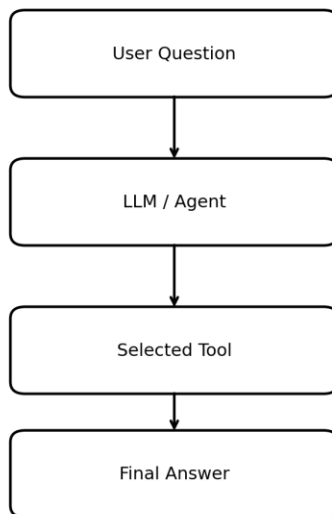
Beginner-to-Intermediate Practical Guide

1. Lab Objective

Build a LangChain agent that has three tools and allows the LLM to decide which tool to use. The lab demonstrates the key transition from a normal LLM application to an AI agent capable of selecting and invoking external capabilities.

- DuckDuckGo – web search
- Wikipedia – reference lookup
- Calculator – custom Python tool

Basic Agent Tool Flow



2. Environment Setup

On Windows, create a project folder and virtual environment:

```
mkdir langchain-agent-demo
cd langchain-agent-demo
python -m venv venv
venv\Scripts\activate
Install the required packages:
```

```
pip install -U langchain langchain-community wikipedia duckduckgo-search
pip install -U langchain-cohere
Set the Cohere API key. In Command Prompt:
```

```
set COHERE_API_KEY=your_cohere_api_key
```

In PowerShell:

```
$env:COHERE_API_KEY="your_cohere_api_key"
```

3. Tool 1 – DuckDuckGo Search

First invoke the search tool directly. At this stage there is no agent: your program explicitly selects the tool.

```

from langchain_community.tools import DuckDuckGoSearchRun

search = DuckDuckGoSearchRun()

result = search.invoke(
    "What are the latest developments in AI agents?"
)

print(result)

```

4. Tool 2 – Wikipedia

```

from langchain_community.tools import WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper

wikipedia_api = WikipediaAPIWrapper(
    top_k_results=2,
    doc_content_chars_max=2000
)

wikipedia_tool = WikipediaQueryRun(
    api_wrapper=wikipedia_api
)

result = wikipedia_tool.invoke("Alan Turing")
print(result)

```

5. Tool 3 – Custom Calculator

The @tool decorator converts a normal Python function into a structured tool that an LLM can understand and invoke.

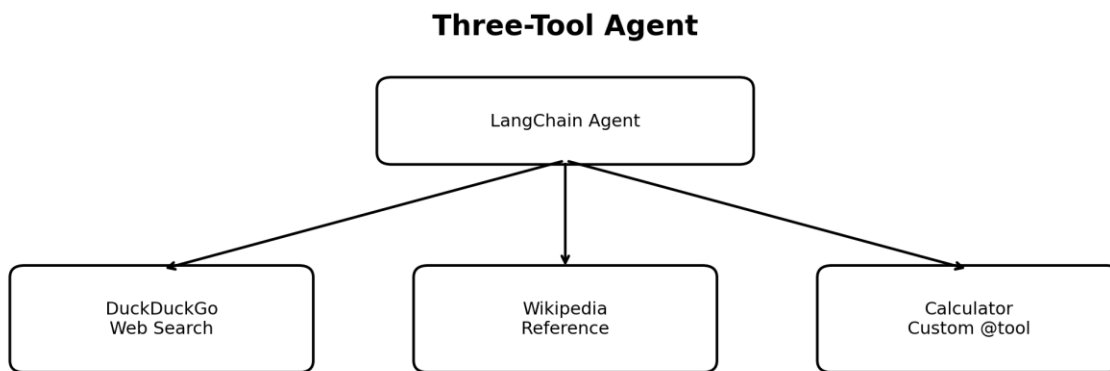
```

from langchain_core.tools import tool

@tool
def calculator(a: float, b: float, operation: str) -> float:
    """Perform arithmetic: add, subtract, multiply, or divide."""
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        if b == 0:
            return "Cannot divide by zero"
        return a / b
    return "Unknown operation"

print(calculator.invoke({
    "a": 25,
    "b": 10,
    "operation": "multiply"
}))

```



6. Create the Three-Tool Agent

The tools are collected into one toolbox. The LLM receives their names, descriptions, and parameter schemas and can select the appropriate tool.

```
tools = [
    search_tool,
    wikipedia_tool,
    calculator
]

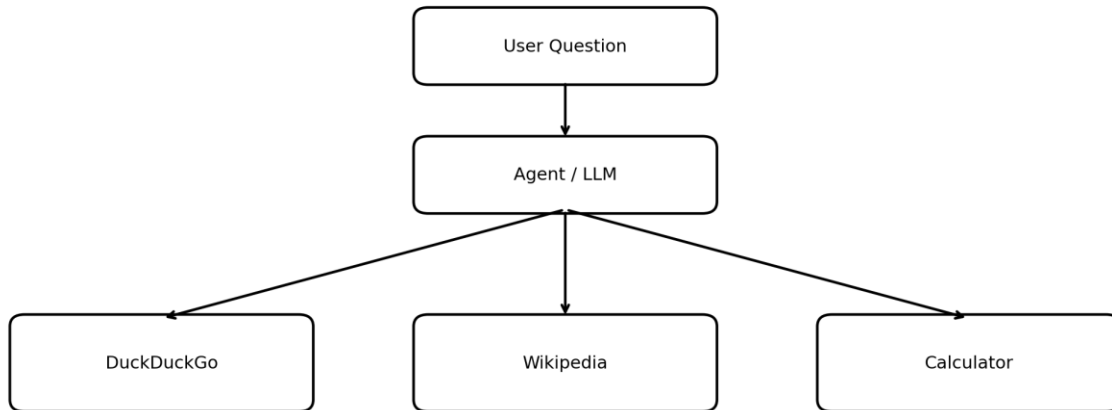
from langchain_cohere import ChatCohere

model = ChatCohere(
    model="command-a-03-2025",
    temperature=0
)

from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=tools,
    system_prompt="""
    You are a helpful AI assistant.
    Use Wikipedia for encyclopedia knowledge.
    Use DuckDuckGo for recent/current information.
    Use the calculator for arithmetic.
    Select tools automatically.
    """
)
```

Automatic Tool Selection



7. Invoke the Agent

```
response = agent.invoke({
    "messages": [
        {
            "role": "user",
            "content": "What is 345 multiplied by 76?"
        }
    ]
})

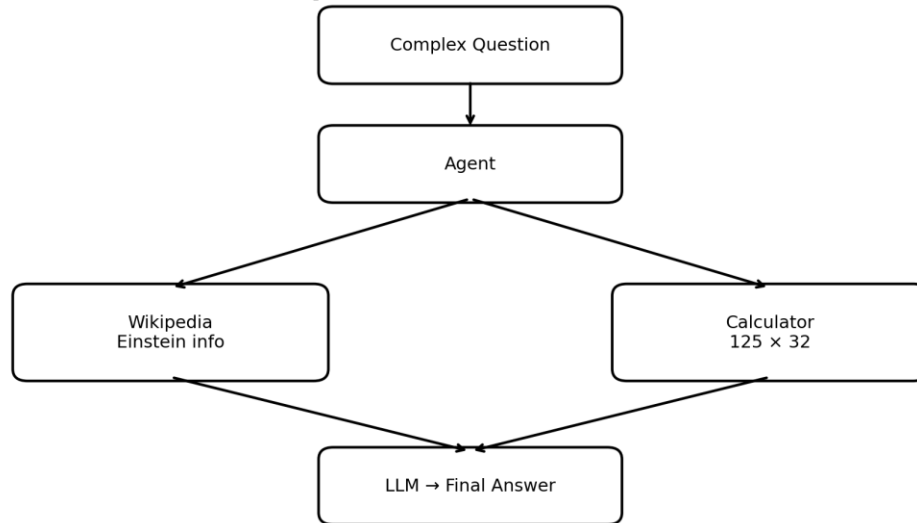
print(response["messages"][-1].content)
```

For this question, the agent should prefer the calculator. For 'Who was Alan Turing?' it can use Wikipedia. For a question about recent developments, it can prefer web search.

8. Multiple Tool Calls

Try: "Who was Albert Einstein and what is 125×32 ?" A capable tool-calling agent can use Wikipedia for the first part and the calculator for the second part before composing one final response.

Multiple Tool Calls in One Task



9. Complete Program

```
from langchain.agents import create_agent
from langchain_cohere import ChatCohere
from langchain_community.tools import DuckDuckGoSearchRun, WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper
from langchain_core.tools import tool
```

```
search_tool = DuckDuckGoSearchRun()
```

```
wikipedia_api = WikipediaAPIWrapper(
    top_k_results=2,
    doc_content_chars_max=2000
)
wikipedia_tool = WikipediaQueryRun(api_wrapper=wikipedia_api)
```

```
@tool
def calculator(a: float, b: float, operation: str) -> float:
    """Perform arithmetic: add, subtract, multiply, or divide."""
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        if b == 0:
            return "Cannot divide by zero"
        return a / b
    return "Unknown operation"
```

```
tools = [search_tool, wikipedia_tool, calculator]
```

```
model = ChatCohere(
    model="command-a-03-2025",
    temperature=0
)
```

```
agent = create_agent(
    model=model,
    tools=tools,
    system_prompt=""
    You are a helpful AI assistant.
```

```

    Use Wikipedia for encyclopedia knowledge.
    Use DuckDuckGo when recent/current information is required.
    Use the calculator for arithmetic.
    Select tools automatically depending on the user's question.
    """
)

question = input("\nAsk the AI Agent a question: ")

response = agent.invoke({
    "messages": [{ "role": "user", "content": question}]
})

print("\nAgent Response:\n")
print(response["messages"][-1].content)

```

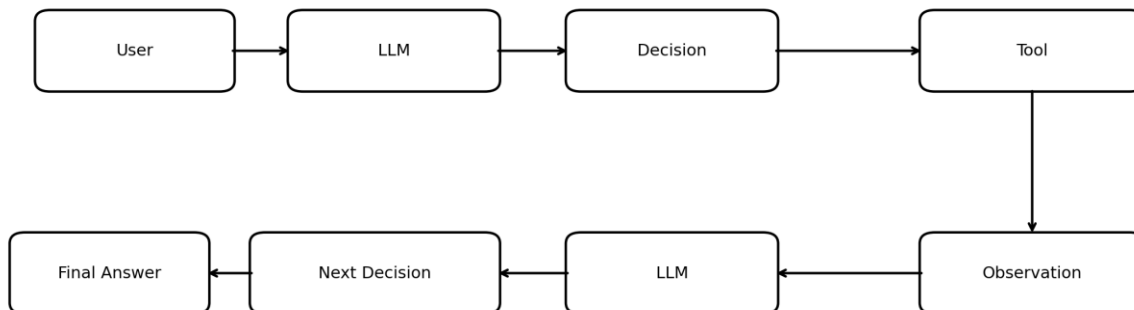
10. Suggested Test Questions

Question	Likely Tool
What is 328×97 ?	Calculator
Who developed the theory of relativity?	Wikipedia
What are recent developments in quantum computing?	DuckDuckGo
Who was Albert Einstein and what is 125×32 ?	Wikipedia + Calculator

11. What Happens Internally?

A normal LLM follows a prompt → response pattern. An agent introduces a decision/action loop. The LLM can decide whether a tool is needed, select it, receive the observation, and then continue reasoning at the application level until a final answer can be produced.

Agentic Decision Loop



12. Key Learning Points

- Direct tool invocation means the programmer chooses the tool.
- Agent tool calling means the LLM chooses from the tools made available to it.
- Tool descriptions and parameter schemas are important because they guide selection.

- An agent may use one tool, multiple tools, or no tool depending on the question.
- Tools allow an LLM application to interact with web search, APIs, databases, files, business systems, and custom Python functions.
- Tool calling is one of the key building blocks connecting LLMs to AI agents and agentic workflows.

13. Next Lab

Extend the agent with a real API tool, a database lookup tool, and a PDF/RAG retriever. Then add conversation state or memory to demonstrate the progression from a simple tool-calling agent toward a richer agentic AI application.